Project Explainer

"When Imbalance Correction Backfires: Robustness of Tabular Imbalance Mitigation Under Minority-Label Contamination" — our project in full: what we studied, how we built it, what we found, and where it sits in the literature

Every number in this document is copied from our own LaTeX source and result files, not paraphrased or reconstructed from memory.

---

1. Project overview

What our project is about

Standard class-imbalance corrections, resampling methods like SMOTE and random oversampling, cost-sensitive reweighting, and, for tabular foundation models, test-time posterior corrections like DistPFN (Lee, 2026), are usually assumed to be safe: they might not always help much, but they are not supposed to actively hurt. That assumption is almost always validated on clean data and then deployed into systems where labels are not clean. We test that assumption directly. If a correction cannot tell a genuine minority example from a mislabeled one, a resampling method will duplicate or synthesize around a mislabeled row exactly as readily as a correctly labeled one, a cost-sensitive method will derive its penalty from a corrupted class ratio, and a posterior-rescaling method will correct against a corrupted prior. In every case, the correction can end up amplifying the mislabeled examples it was built to compensate for, rather than correcting genuine imbalance.

What we did

We audited seven correction methods across two widely deployed tabular model families, XGBoost (Chen & Guestrin, 2016: cost-sensitive weighting, random oversampling, SMOTE) and TabPFN (Hollmann et al., 2023, 2025: threshold correction, DistPFN, DistPFN-T, class-balanced downsampling), each measured against its own uncorrected baseline. We injected two directed, asymmetric label-error mechanisms, majority rows relabeled as minority (Direction A) and minority rows lost into the majority class (Direction B), at four contamination rates plus a shared clean condition, across four imbalance ratios, on ten UCI datasets, with three seeds each. We held total training-set size fixed across all four imbalance ratios (N_{common}, Section 3) specifically so that a change in imbalance ratio could never be confounded with a change in how much training data a method receives. Because raw accuracy alone cannot say whether a correction was worth applying, we introduced five original metrics to judge that question directly: Correction Gain, Correction Harm Rate, Regret, the break-even contamination rate, and the Prior Sensitivity Gap (all defined in Section 3).

Study at a glance

Datasets	10 (UCI Machine Learning Repository)
Model families	XGBoost, TabPFN
Correction methods audited	7, plus 2 uncorrected baselines
Imbalance ratios tested	0.50, 0.20, 0.10, 0.05
Contamination rates tested	0 (clean), 0.05, 0.10, 0.20, 0.30
Corruption directions	2 (majority→minority, minority→majority)
Seeds per condition	3
Total evaluations	**12,960 (derivation in Section 3)**

Row	What it means
Datasets	Ten separate, real-world tabular datasets, all pulled from the UCI Machine Learning Repository, a public archive of datasets used across ML research, spanning finance, security, healthcare, signal processing, astronomy, and chemistry. Ten different datasets were used so no single finding could be explained by one dataset's quirks (the full list is Table I).
Model families	The two underlying machine learning models the corrections are actually applied on top of. XGBoost is a conventional gradient-boosted tree model, retrained on each dataset. TabPFN is a newer tabular foundation model that predicts via in-context learning, reading the training set as a prompt in a single forward pass, with no retraining at all. Each correction method belongs to one family or the other.
Correction methods audited	The actual techniques tested: three for XGBoost (cost-sensitive weighting, random oversampling, SMOTE) and four for TabPFN (threshold correction, DistPFN, DistPFN-T, class-balanced downsampling), seven total. The two uncorrected baselines, xgb_none and tabpfn_none, apply no correction at all and are the reference point every one of the seven is measured against.
Imbalance ratios tested	The fraction of training rows belonging to the minority class, deliberately set to four levels. 0.50 is perfectly balanced; 0.05 is severe imbalance, only 5% minority. Testing all four shows how a correction's behavior changes as imbalance gets worse.
Contamination rates tested	How much of the training labels are deliberately corrupted. 0 is the clean control condition; 0.05 to 0.30 are increasing severities of directed mislabeling.
Corruption directions	The two distinct ways those labels get corrupted. Direction A takes real majority rows and mislabels them as minority (a false alarm). Direction B takes real minority rows and mislabels them as majority (a genuine case lost). Both are tested because a correction robust under one direction is not necessarily robust under the other.
Seeds per condition	A random seed is the starting number fed into the computer's random-number generator, which makes every "random" choice in the experiment fully reproducible: the same seed always produces the exact same sequence of random decisions, and a different seed changes all of them together. Here, a seed controls which rows land in the 80/20 train/test split, which rows get sampled into the controlled training set, which rows get flipped under Direction A or B, and which rows random oversampling, SMOTE, and downsampling choose to duplicate, interpolate, or drop. Every combination of the dimensions above is repeated three times with three different, fixed seeds, {13, 37, 73}, arbitrary numbers with no special meaning, chosen once and reused everywhere, so no result depends on one particular lucky or unlucky realization of all those random choices. This is also a known limitation: three repetitions give only limited precision for Correction Harm Rate and the break-even rate specifically, the two metrics most sensitive to seed-to-seed noise (Section 5, Threats to Validity).
Total evaluations	The grand total number of individual scoring runs across the whole study (10 datasets × 3 seeds × 4 ratios × 9 contamination/direction combinations × 9 methods, plus 3,240 extra oracle-prior diagnostic runs for three of the TabPFN methods, derived in full in Section 3). It establishes how much empirical weight sits behind every later percentage, a reported 20.6% Correction Harm Rate is a count over 160 independent runs for that method and direction, not a guess.

Imbalance ratio (r): what each tested value means

r	Minority share of the training set	Plain meaning
0.50	1 row in every 2	Perfectly balanced, the easiest condition
0.20	1 row in every 5	Mild imbalance
0.10	1 row in every 10	Moderate imbalance
0.05	**1 row in every 20**	**Severe imbalance, the hardest condition tested**

Contamination rate (c): what each tested value means

c	Plain meaning	Actual flip share of n_{min} (Direction A / Direction B)
0 (clean)	No labels corrupted at all, the control condition every method is also scored on	0% / 0%
0.05	Light corruption	5.3% / 5.0%
0.10	Moderate corruption	11.1% / 10.0%
0.20	Heavy corruption	25.0% / 20.0%
0.30	**Severe corruption, the hardest condition tested**	**42.9% / 30.0%**

The two directions define c against different reference points, so the same nominal c is not an identical absolute flip count in each (Section 3 has the full derivation). c is applied after r, on top of whichever imbalance ratio was already set, the two are independent dials, not sequential steps that undo each other.

What we achieved

- Every one of the seven correction methods underperformed doing nothing at all in at least some tested condition. No correction is universally safe.
- The simplest correction tested, a plain threshold shift keyed to the class prior, was the most robust method on discrimination in both corruption directions, beating every more elaborate correction on both Correction Harm Rate and mean Correction Gain.
- The two most mathematically elaborate corrections, DistPFN and DistPFN-T, had the highest Correction Harm Rate of any method under Direction B, and yet were simultaneously the two best-calibrated methods in the entire study, discrimination robustness and calibration quality are not the same property, and one cannot be inferred from the other.
- How often a correction fails depends strongly on the direction of the label error, and that dependence is itself method-specific, a method robust under one direction is not reliably robust under the other.
- The corrupted prior turned out not to be the primary reason TabPFN's posterior-rescaling corrections decline under contamination, an unanticipated negative result: handing them the true, uncorrected prior did not reliably help, and on average even hurt slightly.
- Correction-harm rates across every method and direction range from 3.1% to 29.4%, and all four Friedman omnibus significance tests are significant at $p < 10^{-23}$.
- A stricter, dataset-level statistical test (Section 4) confirmed the central finding while appropriately narrowing two broader claims about which specific method pairs are statistically distinguishable.

The gap in prior work we address

Class imbalance and label noise have each been studied extensively on their own, but rarely together, and never before for tabular foundation models. The literature splits into several distinct groups, each with its own specific limitation relative to what we set out to test. Section 6 gives the full paper-by-paper picture in table form; this section explains each limitation in the prose detail behind it, and exactly what we did on our side to close each one.

1. Classical corrections themselves (SMOTE, random oversampling, cost-sensitive weighting)

LIMITATION SMOTE (Chawla et al., 2002), random oversampling, and cost-sensitive reweighting (Elkan, 2001) were all designed under the implicit assumption that the labels used to compute a resampling ratio, an interpolation neighborhood, or a cost ratio are themselves trustworthy. None of them were built with, or originally evaluated against, the possibility that a fraction of those labels are simply wrong.

WHAT WE DID We took these exact, standard, already-deployed corrections, unmodified, and tested them under two directed, controlled label-error mechanisms at four contamination rates each, then measured precisely how often (Correction Harm Rate) and how badly (Regret) each one backfires relative to doing nothing at all.

2. Van Hulse & Khoshgoftaar (2009), the closest prior audit-style study

LIMITATION This is the one prior work that already asked our kind of question, training over two million classifiers to test how sampling techniques interact with class noise. But it used only classical learners (no tabular foundation models existed yet), and it did not control training-set size across imbalance ratios, so a performance difference in that study could always be explained by sample size rather than by imbalance itself, the two were confounded.

WHAT WE DID We extended the same underlying question to TabPFN and its correction ecosystem (threshold correction, DistPFN, DistPFN-T, downsampling), and we introduced N_{common} (Section 3) specifically to hold total training-set size fixed across every tested imbalance ratio, so that in our results, unlike in that earlier study, a difference in performance can never be attributed to sample size instead of imbalance.

3. Newer noise-robust resampling algorithms: RSMOTE (Chen et al., 2021), WRND (Li et al., 2024)

LIMITATION

Both papers redesign SMOTE's own interpolation step to filter out or down-weight likely-noisy minority points before synthesizing new rows. Both are new algorithms, proposed as replacements, not audits of whether the standard, already-deployed SMOTE that practitioners are using right now remains safe to keep using as it is.

WHAT WE DID

We deliberately did not propose a new algorithm. Our question is narrower and, we think, more immediately useful to a practitioner: does the correction you already have in production remain net-beneficial once some of your labels are wrong, or does it need to be replaced or supplemented? We answer that for the standard, unmodified version directly.

4. Joint imbalance-and-noise training methods: Koziarski et al. (2020), Khanal et al. (2024), Liu et al. (2024)

LIMITATION

These three papers combine cleaning with resampling, active learning with noisy-label training, and class-balanced sample selection, respectively, each a new training-time algorithm for handling imbalance and noise together. Koziarski et al.'s scope is multi-class rather than binary; Khanal et al.'s is specific to medical imaging. None of the three asks whether the imbalance corrections already in widespread use elsewhere remain safe.

WHAT WE DID

We kept our scope to binary classification on tabular data specifically, matching the setting these standard corrections are actually deployed in most widely, and extended the same audit-not-propose question to tabular foundation models, which none of these three papers touch at all.

5. TabPFN itself (Hollmann et al., 2023, 2025) and its internal noise analysis (Hu & Ghelichi, 2026)

LIMITATION

TabPFN's own introduction never examines behavior under class imbalance or label noise, since it predates that question. Hu & Ghelichi come closer, analyzing TabPFN's internal attention patterns directly under synthetic label-quality degradation, and finding its attention stays structured with no correction applied at all, but their analysis targets the base model's internals, not the correction methods practitioners actually layer on top of it in deployment.

WHAT WE DID

We audit the deployed correction methods themselves, threshold correction, DistPFN, DistPFN-T, and downsampling, rather than the uncorrected model's internals, which is the specific layer of the system Hu & Ghelichi's analysis does not reach.

6. DistPFN and DistPFN-T (Lee, 2026), the newest correction methods we audit

LIMITATION

DistPFN was introduced to correct TabPFN's predictions for label shift, but was never evaluated for robustness under directed label contamination, only under clean-data label shift. During our own independent verification of the published formulas against the authors' reference implementation, we additionally found a shape-error bug in the published DistPFN-T reference driver that silently fails for two-class problems, a limitation that exists in the reference code itself, not just in the literature's evaluation scope.

WHAT WE DID

We are the first to test DistPFN and DistPFN-T's robustness under directed, asymmetric label contamination rather than only clean-data label shift, and we implemented and documented a corrected version of the DistPFN-T formula rather than silently reproducing the reference driver's bug.

7. The classical posterior-adjustment idea itself: Saerens et al. (2002)

LIMITATION

This is the original formalization of adjusting a classifier's output probabilities to a new target prior without retraining, decades before tabular foundation models or DistPFN existed. It was formulated for, and only ever tested on, conventional probabilistic classifiers.

WHAT WE DID

Our threshold-correction method is a direct application of this same classical idea to TabPFN, and we are the first to test it, and the more elaborate DistPFN variants built on the same idea, under label noise specifically.

8. Whether balancing helps at all, even on clean data: Elor & Averbuch-Elor (2022)

LIMITATION	This work found that with a sufficiently strong classifier and well-chosen hyperparameters, balancing the training set often provides no benefit over the imbalanced data as is, and can be actively counterproductive. This was tested only on clean, uncorrupted labels.
WHAT WE DID	We show the same "correction may not help" pattern is not just present but grows sharper once label noise is added on top, extending this clean-data finding into the noisy regime it did not test.

9. Theoretical label-noise frameworks: Natarajan et al. (2013), Scott et al. (2013)

LIMITATION	Natarajan et al. formalize class-conditional label noise and prove conditions for recovering a consistent classifier under it. Scott et al. show that asymmetric, class-dependent noise requires different identifiability strategies than symmetric noise. Both are general classifier theory, not tied to, or evaluated against, any specific imbalance-correction method.
WHAT WE DID	Our two directed corruption mechanisms, Direction A and Direction B (Section 3), are a concrete operationalization of exactly this class-conditional, asymmetric framework, applied specifically to test imbalance corrections rather than left as theory.

10. Label-noise surveys: Frénay & Verleysen (2014), Song et al. (2022)

LIMITATION	Both surveys catalog a large body of work on training models to be robust to noisy labels, loss correction, sample selection, label correction, but both treat label noise as a phenomenon to be handled during training in isolation. Neither examines how a label-noise-tolerant training approach interacts with a class-imbalance correction applied alongside it.
WHAT WE DID	We study exactly that unexamined interaction directly, not by proposing a new noise-robust training method, but by auditing whether the imbalance corrections already in widespread use remain net-beneficial once that assumption of trustworthy labels is violated.

Taken together, the shared limitation across all ten of these is the same: whether today's already-widely-used imbalance corrections, evaluated exactly as deployed, remain net-beneficial once a controlled fraction of the labels they operate on are wrong, had not been tested, for any model family, old or new. Closing that gap is what our audit is designed to do.

2. How each correction method actually works

Every method below is walked through on the same small toy training set, so the nine methods can be compared directly. The toy set has 8 majority rows (a "risk score" feature with values 10, 12, 11, 14, 13, 15, 9, 16) and 2 minority rows (values 50 and 55), an 8:2 imbalance. Every number shown was computed directly from the actual formulas in our Methods section, not invented for illustration.

XGBoost family

`xgb_none` — the baseline

Trains directly on the training set exactly as given, imbalance and all. Not a correction; it is the reference every other XGBoost variant is measured against.

`xgb_weighted` — cost-sensitive reweighting

Leaves every row untouched and instead penalizes a mistake on a minority row more heavily during training, in proportion to the observed class ratio (`scale_pos_weight` in XGBoost).

scale_pos_weight = n_{maj} / n_{min}	Result
8 majority rows / 2 minority rows	4.0

A single misclassified minority row now counts as heavily in the loss as four misclassified majority rows.

> Under contamination: this weight is computed from the *observed* class counts. If some of those 8 "majority" rows are actually mislabeled minority rows (Direction A), the ratio itself, and therefore the penalty, is calculated on the wrong numbers without any way to detect it.

`xgb_ros` — random oversampling

Duplicates existing minority rows, chosen with replacement, until the minority count matches the majority count. No new feature values are created, only copies of real rows.

Need	Duplicates drawn (with replacement)	Final minority pool
6 more rows (2→8)	55, 55, 55, 50, 50, 50	50, 55, 55, 55, 55, 50, 50, 50

> Under contamination: if the row valued 50 is actually a mislabeled majority row, ROS copies that single wrong label three or four more times into the training set, with no mechanism to tell it apart from the genuine minority row at 55.

`xgb_smote` — synthetic minority oversampling

Picks a minority row, finds its k nearest minority neighbors (k = min(5, minority count − 1)), and manufactures a new row by interpolating between the two. With only 2 minority rows here, k = 1, so every synthetic point lies somewhere on the line between them.

Interpolating between 50 and 55	6 synthetic points generated
new = 50 + t·(55−50), t~Uniform(0,1)	50.1, 54.1, 54.6, 53.0, 53.6, 52.7

> Under contamination: if 50 is a mislabeled majority row, SMOTE does not just duplicate that error, it manufactures six brand-new, entirely fabricated training rows clustered around it, each one tainted by the same wrong label.

TabPFN family

`tabpfn_none` — the baseline

Takes TabPFN's raw P(minority) for a test row and predicts minority whenever that probability exceeds 0.5. Not a correction; the reference every other TabPFN variant is measured against.

`tabpfn_threshold` — prior-adjusted decision threshold

Changes only where the decision boundary sits, from 0.5 to the observed class-1 prior π. The underlying predicted probabilities are never touched.

π (observed minority rate)	Test row prediction	tabpfn_none decides	tabpfn_threshold decides
2/10 = 0.20	**P(minority) = 0.30**	**majority (0.30 < 0.5)**	**minority (0.30 > 0.20)**

Under contamination: π is the *observed*, possibly corrupted rate, not the true one, so a corrupted training set shifts the threshold itself. Our oracle-prior diagnostic (Prior Sensitivity Gap, Section 3) exists specifically to isolate how much that shift matters.

`tabpfn_distpfn` — posterior rescaling

Rescales TabPFN's predicted probabilities against the observed prior: multiply the raw prediction by the test-batch mean prediction, divide by the prior, then renormalize so the two class probabilities sum to one. The formula, exactly as it appears in our Methods section:

$$\text{prior} = [1-\pi,\ \pi] \qquad \text{adjusted} = \text{Normalize}\left(\frac{p \cdot \bar{p}}{\text{prior}}\right)$$

p is TabPFN's raw prediction for one test row, p̄ is the mean prediction across the whole test batch, and prior is built from π, the same class-1 prior used by tabpfn_threshold.

Input	P(majority)	P(minority)
raw prediction p (this row)	0.700	0.300
batch mean p̄	0.750	0.250
prior (from π = 0.20)	0.800	0.200
adjusted (after renormalizing)	**0.636**	**0.364**

The minority probability moves from 0.300 to 0.364 once rescaled against the prior in this example.

Under contamination: both π and the batch mean prediction can shift together, so a corrupted training set can push this rescaling in either direction, not always toward the correct answer, which is part of why this method's Correction Harm Rate is the highest in our study under Direction B (Section 4).

`tabpfn_distpfn_t` — posterior rescaling with temperature

The same rescaling as DistPFN, but the batch-mean prediction is first passed through a temperature term that adapts to how far the batch mean sits from the prior, before the rescaling step runs. The formula, exactly as it appears in our Methods section:

$$\tau = -\sum_i \bar{p}_i \log(\text{prior}_i) \qquad \bar{p}_T = \text{softmax}(\bar{p}/\tau)$$

τ is an entropy-like term measuring how far the batch-mean prediction sits from the prior. $\bar{p}_T$ is then substituted for $\bar{p}$ in the same DistPFN formula above.

What softmax actually does: it turns any list of numbers into a valid probability distribution, one that is all positive and sums to exactly one, by exponentiating every entry and then dividing each one by the total of all the exponentials. Dividing by τ first, then applying softmax, is what lets a small τ sharpen the two probabilities further apart and a large τ pull them back toward 50/50.

Step	P(majority)	P(minority)
$\bar{p}$ (batch mean, before temperature)	0.750	0.250
$\tau = -\sum \bar{p}_i \log(\text{prior}_i)$	0.570	
$\bar{p}/\tau$	1.316	0.439
$\exp(\bar{p}/\tau)$	3.728	1.551
sum of exponentials	5.278	
$\bar{p}_T$ = softmax($\bar{p}/\tau$) = exp($\bar{p}/\tau$) ÷ sum	**0.706**	**0.294**

Step	Value
final adjusted prediction ($\bar{p}_T$ substituted into the DistPFN formula)	**[0.584, 0.416]**

In this example the temperature term pushes the minority probability further, from DistPFN's 0.364 to 0.416, than the untempered version alone.

> Under contamination: the temperature term is itself computed from the same observed batch mean and prior, so it inherits the same sensitivity to corruption that DistPFN has, rather than correcting for it.

`tabpfn_downsample` — class-balanced downsampling

The family's one data-level correction. Identifies whichever class currently has more observed rows and randomly drops rows from it, without replacement, until it matches the smaller class's count. TabPFN then reads only this balanced subset as its prompt.

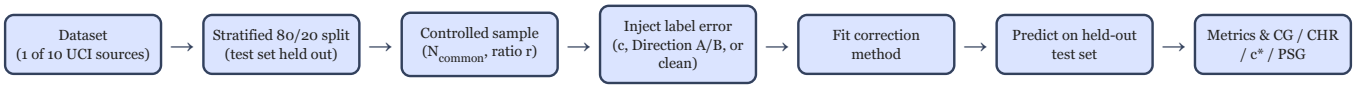
Before	After downsampling
8 majority + 2 minority = 10 rows	2 majority (randomly kept) + 2 minority = 4 rows

> Under contamination: because it identifies the larger class from the *observed* labels directly rather than assuming a fixed encoding, it keeps working even when contamination pushes the observed minority count above the observed majority count at r = 0.50 under Direction A, exactly the condition where a fixed-encoding assumption would silently downsample the wrong class instead.

3. Experimental design: formulas and worked derivations

Every formula our study relies on, typeset exactly as it appears in our Methods section, with a When / Where / How / Why reading grounded in the actual experiment and real numbers computed from our own data.

The pipeline, end to end



Reproduces our Figure 1 exactly (Methods §4, Overview). The test set is held out once, at the initial split, and stays fixed and clean for the entire run. Only the training side is resampled and corrupted before each correction method is fit and then scored against that same held-out test set.

WHEN	This sequence runs once for every (dataset, seed, imbalance ratio, contamination rate, direction) combination, in that order, from a fresh stratified split through to final scoring.
WHERE	Methods §4, "Overview," Figure 1.
HOW	Each box is a hard boundary: nothing downstream can leak information back upstream. The test set, frozen at the second box, is never touched again by any later step.
WHY	Keeping the test set fixed and clean while only the training side is resampled and corrupted is what makes every method's score comparable to every other method's score on the identical evaluation, and is what makes a Correction Gain of zero mean "no different from doing nothing" rather than "evaluated on an easier or harder test set."

What is a "stress cell"? It is the name we give to one single trip through the pipeline above, one specific combination of one dataset, one seed, one imbalance ratio r, one contamination rate c, and one corruption direction. Each stress cell fixes exactly one corrupted training set, built once, then every one of the nine methods (seven corrections plus two uncorrected baselines) is fit on that same corrupted set and scored against that same held-out test set, so all nine can be compared fairly on identical ground. "Stress" refers to what that training set is being put through: a controlled amount of imbalance and a controlled amount of directed label corruption, to see whether each correction still holds up under it. There are 1,080 stress cells in total (10 datasets × 3 seeds × 4 ratios × 9 contamination/direction combinations, Section 3 below), and 12,960 individual method evaluations across all of them, since most stress cells are scored by more than one method and three prior-dependent methods are scored twice each.

Methodology at a glance: every stage of our pipeline in one table

The diagram above shows the sequence, this table is the same pipeline expanded with every parameter, every tested value, and where each stage is defined in our Methods section, so the whole working mechanism is in one place.

Stage	What happens	Parameters / values we tested	Defined in
1. Dataset selection	We pick one of our ten UCI tabular classification datasets as the source pool for this run.	10 datasets (Banknote Authentication, Blood Transfusion, South German Credit, Spambase, Breast Cancer Wisconsin, Ionosphere, Haberman's Survival, Statlog Heart, HTRU2, QSAR Biodegradation), Table I below.	Methods §4.1, Datasets.
2. Stratified 80/20 split	We split that dataset once into a training pool and a held-out test set, stratified so both splits keep the dataset's natural class balance. The test set is frozen here and never touched again.	80% train pool / 20% test, one split per (dataset, seed) pair, repeated for 3 seeds.	Methods §4, Overview.
3. Controlled sample	We draw a fixed-size training sample from the training pool at a target minority ratio r, using N_{common} so the total training size never changes across ratios.	$r \in \{0.50, 0.20, 0.10, 0.05\}$; N_{common} computed per dataset per seed (Table I).	Methods §4.2, Controlled Imbalance.

4. Label-error injection	We flip a controlled share of minority-class labels in the training sample only, using one of two directed mechanisms, or we leave the sample clean.	$c \in \{0, 0.05, 0.10, 0.20, 0.30\}$; Direction A or Direction B; 9 contamination/direction combinations per ratio (the clean condition counted once, not once per direction).	Methods §4.3, Directed Label-Error Mechanisms.
5. Fit correction method	We fit each of our nine methods (two uncorrected baselines plus seven corrections, split across the XGBoost and TabPFN families) on that same corrupted training sample.	XGBoost family: xgb_none, xgb_weighted, xgb_ros, xgb_smote. TabPFN family: tabpfn_none, tabpfn_threshold, tabpfn_distpfn, tabpfn_distpfn_t, tabpfn_downsample. All nine share the identical training sample for a given stress cell.	Methods §4.4, Correction Methods (Section 2 above walks through each mechanism).
6. Predict on held-out test set	Each fitted method scores the same frozen, clean test set from stage 2, so no method is ever evaluated on an easier or harder test set than any other.	One prediction pass per method per stress cell; three prior-dependent TabPFN methods (threshold, distpfn, distpfn_t) additionally predict once more under an oracle prior for the PSG diagnostic.	Methods §4, Overview.
7. Evaluation metrics	We convert each method's predictions into our ten evaluation metrics covering discrimination, minority-class performance, and calibration.	Balanced accuracy, worst-class recall, AUPRC, macro F1, minority precision, minority recall, AUROC, Brier score, log loss, ECE (all ten defined in full below).	Methods §4.5, Evaluation Metrics.
8. Contribution metrics	We compare each correction's score against its matched uncorrected baseline on the identical stress cell to isolate what the correction itself contributed.	Correction Gain (CG), Correction Harm Rate (CHR), Regret, and Prior Sensitivity Gap (PSG, prior-dependent methods only).	Methods §4.6, Contribution Metrics.
9. Repeat across seeds	We repeat stages 2 through 8 with three different random seeds so no result depends on one particular split or one particular set of flipped rows, then average.	Seeds {13, 37, 73}, 3 per stress cell.	Methods §4.8, Implementation and Reproducibility.
10. Statistical testing	Within each model family and direction, we run a Friedman omnibus test across methods, then, if significant, pairwise Wilcoxon signed-rank tests with Holm correction, at two blocking levels.	Blocked by (dataset, r, c) (n=160) and, more conservatively, by dataset alone (n=10); significance threshold $p<0.05$ throughout.	Methods §4.7, Statistical Testing.

Stages 2 through 8 together define one stress cell (see the callout above); stages 9 and 10 happen after every stress cell has been run, across the whole study.

Table I: dataset characteristics

Why ten different datasets, not one? If we had tested on a single dataset and a correction method came out looking robust, we would not be able to tell whether that reflects a real property of the method or just a coincidence of that one dataset's row count, feature count, domain, or natural class balance. We chose these ten specifically so that no single result could be attributed to one dataset's idiosyncrasies (Methods §4.1, Datasets): they deliberately do not resemble each other, spanning finance (South German Credit), security (Spambase), healthcare (Breast Cancer Wisconsin, Statlog Heart, Haberman's Survival), signal processing (Ionosphere), astronomy (HTRU2), chemistry (QSAR Biodegradation), and forensics (Banknote Authentication, Blood Transfusion), ranging from 270 to 17,898 rows and from 3 to 57 features. When a finding, such as tabpfn_threshold having the lowest Correction Harm Rate, holds up after pooling across all ten of these unrelated datasets (Section 4), that is far stronger evidence of a real property of the method than the same finding on any one dataset alone, since it is much harder to explain away as a coincidence of that dataset's particular shape.

Dataset	Rows	Feat.	N_{common}	Minority meaning
Banknote Authentication	1,372	4	641	Forged banknote
Blood Transfusion	748	4	284	Blood donated in target period
South German Credit	1,000	20	480	Bad credit risk
Spambase	4,601	57	2,347	Spam email

Breast Cancer Wisconsin	569	30	300	Malignant diagnosis
Ionosphere	351	34	189	No ionospheric structure
Haberman's Survival	306	3	128	Death within 5 years
Statlog Heart	270	13	126	Heart disease present
HTRU2	17,898	8	2,622	Pulsar candidate
QSAR Biodegradation	1,055	41	568	Ready biodegradable

Dataset names link to their UCI Machine Learning Repository page. N_{common} is the controlled training size every stress cell actually draws from, not the raw row count (see the worked N_{common} derivation just below).

Controlled training size N_{common}

$$\text{feasible_n}(r) = \min\left(\frac{n_{\min}}{r},\ \frac{n_{\max}}{1-r}\right)$$

The largest total training size achievable at one ratio **r** is bounded by whichever class (minority or majority) runs out first.

$$N_{\text{common}} = \left\lfloor \min_{r \in \{0.50,\, 0.20,\, 0.10,\, 0.05\}} \text{feasible_n}(r) \right\rfloor$$

N_{common} is the tightest of these four constraints across all tested ratios, floored to a whole number of rows. This is the number reported in Table I for every dataset.

WHEN	Computed once per dataset, per seed, immediately after the 80/20 train/test split and before anything else. It is the first quantity fixed in the whole pipeline, before imbalance ratio or label corruption are ever applied.
WHERE	Methods §4.2, "Controlled Imbalance: A Common Training Size." Used as the fixed total training size for all four tested ratios (r = 0.50, 0.20, 0.10, 0.05) on that dataset and seed.
HOW	For each candidate ratio, feasible_n(r) computes the largest total size achievable, capped by whichever class runs out first. N_{common} takes the smallest of the four resulting values, the tightest constraint, since that is the one size every ratio can actually reach.
WHY	Comparing a model trained at r = 0.50 against one trained at r = 0.05 is only informative about imbalance if both use the same total amount of training data. Without this step, a performance difference could be explained by sample size instead of imbalance.

Worked example: Banknote Authentication, seed 13 (reproduces our reported N_{common} = 641)

Quantity	Value
Minority rows in the 80% training pool (n_{min})	488
Majority rows in the 80% training pool (n_{max})	609
feasible_n(0.50) = min(488/0.50, 609/0.50)	976
feasible_n(0.20) = min(488/0.20, 609/0.80)	761
feasible_n(0.10) = min(488/0.10, 609/0.90)	677
feasible_n(0.05) = min(488/0.05, 609/0.95) ← binding constraint	**641**
N_{common} = floor(min of the four)	**641**

How each ratio's training set is actually built from N_{common} Once N_{common} is fixed, building any one of the four ratios is just splitting that same fixed total differently, never drawing a new total each time. For ratio r we draw round(r × N_{common}) minority rows and the remaining N_{common} rows from the majority class, both sampled without replacement from the training pool, then shuffled together. No row is created, deleted, or altered, we only choose how many of each class to include, and the rest of the pool is simply not used for that stress cell.

Banknote Authentication, seed 13: how N_{common} = 641 splits at each tested ratio

r	Minority rows drawn round(r×641)	Majority rows drawn 641−minority	Note
0.50	320–321	320–321	Both classes undersampled well below what the pool has (488 / 609 available)
0.20	128	513	Minority cut down harder to reach the lower ratio
0.10	64	577	Minority cut down further still
0.05	**32**	**609**	**Majority draw uses every one of the 609 available rows, the binding case that set N_{common} itself**

The two directed corruption mechanisms

$$c = \frac{k}{n_{\min} + k} \implies k = \frac{c}{1-c}\, n_{\min} \quad \text{(Direction A)}$$

Direction A defines the contamination rate **c** as the fraction of the *observed* minority-labeled pool that is actually mislabeled majority rows.

$$k = c \cdot n_{\min} \quad \text{(Direction B)}$$

Direction B instead defines **c** directly against the true minority count. These are two different reference points, not the same manipulation flipped in sign.

WHEN	Applied once per stress cell, right after N_{common}'s controlled sample is drawn for a given ratio r, at each of the four non-zero contamination rates and in each of the two directions, before any correction method is fit.
WHERE	Methods §4.3, "Directed Label-Error Mechanisms." Every stress cell picks exactly one of these two formulas to decide how many rows to flip and which pool to flip them from.
HOW	Direction A solves for the flip count k so that the pool a downstream method actually *observes* as "minority" is c-fraction contaminated. Direction B instead sets k directly as a fraction of the true minority count. Both then draw k rows uniformly at random from the relevant pool and relabel them.
WHY	To model two distinct, realistic label-error processes rather than one symmetric random flip. Direction A models false alarms entering the minority class (e.g. a healthy case wrongly flagged). Direction B models genuine minority cases being lost into the majority class (e.g. a real case wrongly cleared). Testing both separately is what lets us show a correction's robustness under one does not predict its robustness under the other.

Direction A vs. Direction B: flip rate relative to n_{min}, at every tested contamination rate

c (nominal)	Direction A: c/(1−c)	Direction B: c	Gap
0.05	5.3%	5.0%	0.3 pts
0.10	11.1%	10.0%	1.1 pts
0.20	25.0%	20.0%	5.0 pts
0.30	**42.9%**	**30.0%**	**12.9 pts**

Worked example: contaminating the r = 0.20 sample above at c = 0.30 The n_{min} in both formulas is the minority count already in the r-adjusted sample built above (128), not the pool-level n_{min} (488) from the N_{common} step, since corruption is applied after the ratio is already fixed, on top of whichever sample that ratio produced.

Banknote Authentication, r = 0.20 sample (128 minority / 513 majority rows), contaminated at c = 0.30

Direction	Formula	k (rows flipped)	Drawn from	Result
A (false alarms)	$k = c/(1-c) \times n_{min}$ = 0.30/0.70 × 128	54.86 → **55**	The 513 majority rows	55 majority rows relabeled minority. Sample now shows 128+55=183 rows labeled minority, of which 55/183 = 30.0% are fake, matching c by construction.
B (lost cases)	$k = c \times n_{min}$ = 0.30 × 128	38.4 → **38**	The 128 minority rows themselves	38 minority rows relabeled majority. Only 128−38=90 true minority rows remain correctly labeled, 38/128 = 29.7% ≈ c.

Same starting sample, same nominal c, but Direction A draws its 55 flips from the 513-row majority pool and adds fake minority labels on top of the real 128, while Direction B draws its 38 flips from the 128-row minority pool itself and deletes real minority labels from within it, which is why the two directions are not the same manipulation with a sign flipped.

The ten evaluation metrics

Every method, on every stress cell, is scored against the fixed, clean test set with these ten metrics. They measure how good a method is in isolation, raw performance, with no reference to any other method. They are the raw material the five contribution metrics below are built from, CG, CHR, Regret, and PSG can each be computed on any one of these ten (the headline numbers in Results use balanced accuracy; Brier score and ECE get their own separate gain figures for the calibration results specifically).

Metric	What it measures	How it is computed	Better is
Balanced accuracy	Overall discrimination, corrected for class imbalance so the majority class cannot dominate the score	Average of the two per-class recalls: (true positive rate + true negative rate) / 2	**Higher**
Worst-class recall	Whether one class collapses even while the average looks fine	The lower of the two per-class recalls, $\min(\text{recall}_{\text{majority}}, \text{recall}_{\text{minority}})$	**Higher**
AUPRC	Precision/recall trade-off, focused on how well the minority class specifically is served across every possible decision threshold	Area under the precision-recall curve	**Higher**
Macro F1	Balance of precision and recall, weighting both classes equally regardless of size	Unweighted average of each class's own F1 score (the harmonic mean of that class's precision and recall)	**Higher**
Minority precision	Of everything the model flags as minority, how much of it really is	True positives / (true positives + false positives), minority class only	**Higher**
Minority recall	Of everything that really is minority, how much the model actually catches	True positives / (true positives + false negatives), minority class only	**Higher**
AUROC	Whether the model can rank a random true positive above a random true negative, independent of any single decision threshold	Area under the ROC curve (true positive rate vs. false positive rate) across every threshold	**Higher**
Brier score	Combined discrimination and calibration of the predicted probabilities themselves, not just the final yes/no decision	Mean squared difference between the predicted probability and the actual outcome (0 or 1)	**Lower**
Log loss	Same idea as Brier score, but penalizes a confident wrong prediction much more harshly than a hedged one	Negative log-likelihood of the true label under the model's predicted probability	**Lower**
ECE	Calibration specifically, do predicted probabilities match the true observed frequencies, independent of whether the final decision was correct	Sort predictions into ten equal-width probability bins, average the gap between each bin's mean predicted probability and its actual observed positive rate	**Lower**

All ten are computed identically for every method and condition using scikit-learn's implementations (Pedregosa et al., 2011), so no method is scored on a different basis than any other. Worst-class recall exists specifically because balanced accuracy is itself an average, a method could score 100% recall on the majority class and 40% on the minority class and still show a misleadingly reasonable 70% balanced accuracy, worst-class recall is what catches that collapse. Brier score, log loss, and ECE are the three calibration-facing metrics in this list, distinct from the other seven, which are discrimination-facing, this split is exactly what lets Results show a correction can excel at one axis while failing badly at the other (Section 5, "Accuracy and calibration answer different questions").

Why only three of these ten actually appear in Results All ten are computed and stored for every one of the 12,960 evaluations, but our reported figures and tables are built on only three: balanced accuracy, Brier score, and ECE. This was not decided after seeing which numbers looked best, it follows a three-tier split we wrote into our own research plan before running a single experiment: **primary** (balanced accuracy, worst-class recall, AUPRC, macro F1, discrimination metrics that largely move together), **diagnostic** (minority precision, minority recall, AUROC, explicitly planned as secondary sanity checks, not headline evidence on their own), and **reliability** (Brier score, log loss, ECE, attached to a hypothesis we wrote down in advance that calibration would reveal correction harm that discrimination metrics alone would miss, which is exactly what Results §4 found for the DistPFN pair). We report balanced accuracy as the one representative of the primary tier, since the other three in that tier are highly correlated with it and would mostly repeat the same result rather than add new information, and we report two of the three reliability metrics, Brier score and ECE, because that pairing is where our results genuinely diverge from the discrimination story. Log loss is computed for every run and worked out mechanically in this document (just below), but its Pearson correlation with Brier score across our full result set is 0.949, so like the primary-tier metrics it would mostly repeat the Brier score story rather than add independent evidence, which is why it never actually appears with a reported number in Results either, this document's own explanation of it is more thorough than the paper's actual use of it. The three diagnostic metrics, log loss, and the three unreported primary-tier metrics are all sitting in our raw results if a check is ever needed, they were simply never the metrics our written claims are built on.

This reasoning now also lives in the paper itself, not just here. We added a paragraph to Methods §4.5, Evaluation Metrics, stating this same tiering directly: balanced accuracy carries the discrimination claims because worst-class recall, AUPRC, and macro F1 correlate with it too strongly to add independent evidence (Pearson correlations of 0.966, 0.892, and 0.987 respectively, checked directly against our own results before writing the sentence, and deliberately not written as "r = ..." in that paragraph since r already means imbalance ratio everywhere else in Methods), minority precision, minority recall, and AUROC are named as secondary diagnostics, and Brier score and ECE are named as the two reliability metrics results are actually built on. Log loss is stated as computed but not separately reported because it also correlates strongly with Brier score, the same qualitative phrasing used for the primary tier rather than a bare, uncited decimal. We caught that inconsistency ourselves, an early draft of that sentence cited 0.949 directly, a real number we had verified but never saved to a citable results file, sitting next to three qualitative claims with no number at all, so we matched the paragraph's own existing style instead of leaving one unreferenced figure standing out. What was a silent gap when this document was first written is now a stated, deliberate scope decision in the paper a reader can actually see.

The three calibration-facing scores, worked out on one shared example

Brier score, log loss, and ECE all grade a predicted *probability* rather than a final yes/no decision, but they penalize a wrong probability in three different ways. The same five toy predictions are scored by all three below so the difference is visible directly, not just described in words.

Shared toy predictions for this example

Row	True label y	Predicted P(minority) p
1	1 (minority)	0.90
2	0 (majority)	0.20
3	1 (minority)	0.40
4	0 (majority)	0.10
5	0 (majority)	0.60

Brier score: mean squared error between p and y

FORMULA Brier = mean((p − y)²) across every row, where y is 1 or 0.

WORKED OUT Row 1: (0.90−1)²=0.01. Row 2: (0.20−0)²=0.04. Row 3: (0.40−1)²=0.36. Row 4: (0.10−0)²=0.01. Row 5: (0.60−0)²=0.36. Mean = (0.01+0.04+0.36+0.01+0.36)/5 = **0.156**.

Notice rows 3 and 5 contribute the identical penalty, 0.36, even though row 3 was a minority row scored only 0.40 (an unconfident miss, arguably "almost right") and row 5 was a majority row scored 0.60 (a confident-leaning wrong call in the other direction). Brier score treats being wrong by the same margin the same way regardless of which direction the error runs.

Log loss: negative log-likelihood of the true label

FORMULA Log loss = mean(−[y·log(p) + (1−y)·log(1−p)]) across every row.

WORKED OUT Row 1: −log(0.90)=0.105. Row 2: −log(1−0.20)=0.223. Row 3: −log(0.40)=0.916. Row 4: −log(1−0.10)=0.105. Row 5: −log(1−0.60)=0.916. Mean = (0.105+0.223+0.916+0.105+0.916)/5 = **0.453**.

Log loss uses −log instead of a squared difference, which grows much faster as a confident prediction lands on the wrong side. A prediction of 0.60 on an actual majority row already costs 0.916, if that same wrong call had been made with 95% confidence instead of 60%, the squared-error penalty (Brier) would only rise to 0.9025, but the log-loss penalty would jump to 3.0, more than three times as harsh. This is why log loss is described as punishing confident wrong predictions especially severely, where Brier score treats every error more evenly.

ECE: does the predicted probability match the observed frequency

FORMULA Sort predictions into equal-width probability bins (our study uses ten, 0–10%, 10–20%, and so on). In each bin, compare the bin's mean predicted probability to the actual fraction of true positives it contains, then average that gap across bins, weighted by how many rows fall in each one.

WORKED OUT (TWO BINS, FOR A COMPACT EXAMPLE) Bin [0, 0.5): rows 2, 3, 4 (p = 0.20, 0.40, 0.10). Mean predicted = 0.233. Observed rate = 1 of 3 rows is actually minority = 0.333. Gap = 0.100, weight = 3/5. Bin [0.5, 1.0]: rows 1, 5 (p = 0.90, 0.60). Mean predicted = 0.750. Observed rate = 1 of 2 rows is actually minority = 0.500. Gap = 0.250, weight = 2/5. ECE = 0.6×0.100 + 0.4×0.250 = **0.160**.

ECE answers a question neither Brier score nor log loss answers directly: of every time the model said "about 75% minority," was the true rate in that group actually close to 75%? Here it was only 50%, the model's high-confidence bin is overconfident. A method can have a perfectly reasonable Brier score or log loss on average while still being badly miscalibrated in exactly this way, which is why the study tracks all three rather than only one.

The five contribution metrics

$$\mathrm{CG} = S_{\text{corrected}} - S_{\text{uncorrected}}$$

Correction Gain: the corrected method's score minus the uncorrected baseline's score, on the same condition.

$$\mathrm{CHR} = \frac{\#\{\text{cells with CG} < 0\}}{\#\{\text{cells tested}\}}$$

Correction Harm Rate: the fraction of tested conditions where a correction's Correction Gain is negative, i.e. it did worse than doing nothing. This is a comparison across experimental conditions, not a per-prediction error rate.

$$\mathrm{Regret} = \max(0, \quad -\mathrm{CG})$$

Regret: how large the harm is when a correction does fail; zero whenever it does not.

$$\mathrm{PSG} = S_{\text{oracle prior}} - S_{\text{observed prior}}$$

Prior Sensitivity Gap: the diagnostic oracle-prior score minus the real, deployable observed-prior score.

WHEN	Computed only after a correction method has already been fit and scored against the fixed test set for a stress cell, the last step of the pipeline. CG, CHR, and Regret are then pooled across stress cells to produce the headline per-method numbers in Results; PSG is computed once per stress cell for the three prior-dependent methods and averaged the same way.
WHERE	Methods §4.6, "Contribution Metrics." CG/CHR/Regret apply to all seven corrections, each against its matched uncorrected baseline. PSG applies only to the three prior-dependent TabPFN methods, since only they have a diagnostic oracle-prior variant to compare against.

HOW	CG is a direct difference in score. CHR counts how often that difference is negative across every tested cell. Regret keeps only the negative part of CG. PSG compares the same method's oracle- and observed-prior variants on the identical condition.
WHY	Raw accuracy alone does not say whether applying a correction was worth it, only a direct, same-condition comparison against doing nothing does. The four judgments separate whether it helps (CG), how often it fails (CHR), how badly (Regret), and whether a wrong prior explains the failure (PSG).

Which of the nine methods each metric applies to

Method	CG / CHR / Regret	PSG
xgb_none, tabpfn_none	baseline (not scored)	—
xgb_weighted, xgb_ros, xgb_smote	yes	—
tabpfn_downsample	yes	—
tabpfn_threshold, tabpfn_distpfn, tabpfn_distpfn_t	**yes**	**yes**

Scale of our study: where 12,960 evaluations comes from

$$10 \times 3 \times 4 \times 9 = 1{,}080 \text{ stress cells}$$

10 datasets × 3 seeds × 4 imbalance ratios × 9 contamination/direction combinations.

$$9 \text{ methods} \times 1{,}080 = 9{,}720 \text{ evaluations}$$

Nine methods (seven corrections plus two uncorrected baselines), scored once per cell.

$$3 \text{ prior-dependent methods} \times 1{,}080 = 3{,}240 \text{ additional evaluations}$$

The three prior-dependent TabPFN methods are additionally scored a second time with a diagnostic oracle prior.

$$9{,}720 + 3{,}240 = 12{,}960$$

The two add up to exactly the 12,960 figure we reported.

The nine contamination / direction combinations tested per (dataset, seed, ratio)

c	Direction	Label used in the data
0.00	shared, both directions	clean
0.05	A	majority_to_minority
0.05	B	minority_to_majority
0.10	A	majority_to_minority
0.10	B	minority_to_majority
0.20	A	majority_to_minority
0.20	B	minority_to_majority
0.30	A	majority_to_minority
0.30	B	minority_to_majority

Natural class prevalence versus the tested ratios

$$\text{HTRU2 natural prevalence} = 9.16\% \quad \text{vs.} \quad r = 0.10 = 10\%$$

Computed directly from HTRU2's raw class counts: its natural minority prevalence sits almost exactly on top of the r = 0.10 condition.

WHEN	A fixed property of each raw dataset, but experimentally relevant at scoring time, since the test partition is drawn once and never resampled, so it always reflects this natural rate regardless of what ratio r the training side is pushed to.
WHERE	Methods §4.1, "Datasets," and disclosed as a limitation in Threats to Validity (Section 5).
HOW	Minority row count divided by total row count, from each dataset's raw class counts.
WHY	The test set's balance never changes while training is deliberately forced to r. At the two lowest tested ratios this creates an unavoidable train/test prior mismatch on top of imbalance itself.

Natural minority prevalence, all ten datasets, sorted low to high

Dataset	Prevalence
HTRU2	**9.16%**
Blood Transfusion	23.80%
Haberman's Survival	26.47%
South German Credit	30.00%
QSAR Biodegradation	33.74%
Ionosphere	35.90%
Breast Cancer Wisconsin	37.26%
Spambase	39.40%
Statlog Heart	44.44%
Banknote Authentication	44.46%

4. **Results**

Every table below reproduces the actual numbers we reported in Results.

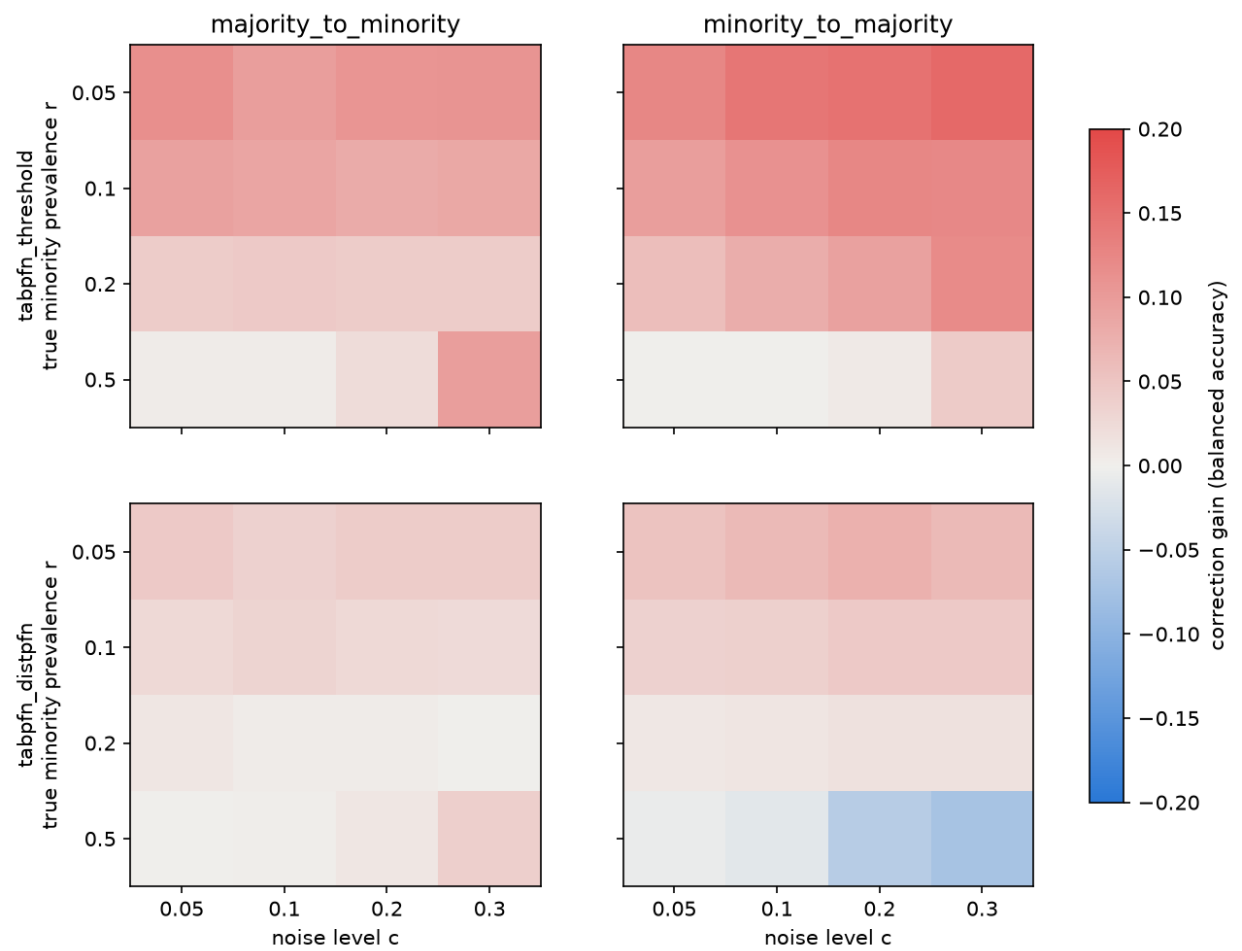
Overall robustness: Correction Harm Rate and mean Correction Gain

Correction Harm Rate (CHR) and mean Correction Gain (CG, balanced accuracy) by method and direction, over noisy stress cells (c > 0)

Method	CHR		mean CG	
	Dir. A	Dir. B	Dir. A	Dir. B
xgb_weighted	0.169	0.094	0.020	0.029
xgb_ros	0.188	0.125	0.018	0.030
xgb_smote	0.213	0.081	0.024	0.040
tabpfn_threshold	**0.081**	**0.031**	**0.067**	**0.090**
tabpfn_distpfn	0.213	0.294	0.021	0.020
tabpfn_distpfn_t	0.213	0.294	0.026	0.023
tabpfn_downsample	0.206	0.075	0.047	0.071

`tabpfn_threshold` (highlighted) is the most robust method by both measures in both directions. Under Direction A, four methods cluster within a point of each other for the highest CHR. Under Direction B, the DistPFN pair separates clearly from the rest, tied at 29.4%, more than double the next-highest method.

Requiring at least a 0.01 balanced-accuracy-point drop before counting a cell as harmful (a magnitude-based sensitivity check) changes the picture: `tabpfn_threshold`'s Direction A CHR falls to 0%, and every other method's harm rate drops by half or more except `tabpfn_downsample`, which stays at 10.6%, the one method still failing meaningfully often under Direction A. Under Direction B the DistPFN pair's separation gets clearer still, their harm rate stays at 16.3% and 13.1%, four to five times any other method's.



Correction gain (balanced accuracy) across imbalance ratio r and noise level c, for the most robust corrected method (tabpfn_threshold, top) and the least robust (tabpfn_distpfn, bottom). Blue indicates the correction hurt relative to the uncorrected baseline at that cell, red indicates it helped.

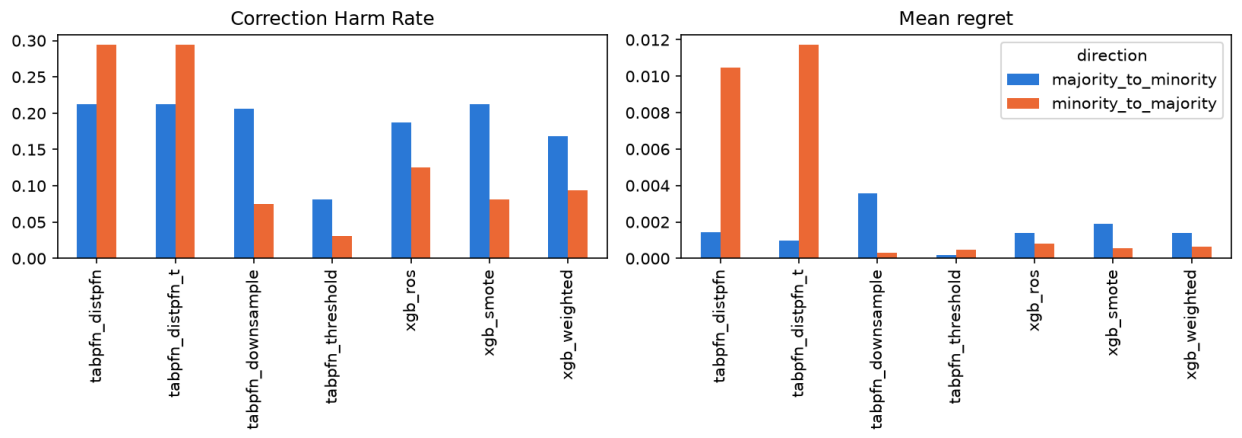
WHEN	Built after every stress cell has been scored, by averaging Correction Gain across all ten datasets for each (method, r, c, direction) combination, so each of the 16 cells in a panel represents 30 underlying runs (10 datasets × 3 seeds).
WHERE	Results §4 (Overall Correction Robustness), Figure 2.
HOW	A 4×4 grid per method (four ratios r by four noise levels c), colored on a diverging red-blue scale centered at zero gain, split into two columns, one per corruption direction.
WHY	A single CHR number hides *where* a method fails. This figure shows tabpfn_threshold stays red (helpful) across the entire grid in both directions, while tabpfn_distpfn's harm concentrates specifically at r=0.50 combined with the highest tested contamination, it is not uniformly bad, it fails in one identifiable corner of the condition space.

How to read this figure, panel by panel

The four panels are two methods × two directions: top row is tabpfn_threshold (our most robust correction), bottom row is tabpfn_distpfn (our least robust). Left column is Direction A (majority rows mislabeled as minority), right column is Direction B (minority rows mislabeled as majority).

Inside each panel, the y-axis is imbalance ratio r, running 0.05 to 0.5 top to bottom, so the **top row of every grid is the most severely imbalanced condition** and the bottom row is nearly balanced. The x-axis is noise level c, running 0.05 to 0.3 left to right. Each of the 16 small squares is therefore one specific (r, c) combination, and its color is the actual correction gain at that cell, averaged over 10 datasets × 3 seeds so no single square is a fluke of one dataset. Red means the correction helped, blue means it hurt, pale/white means essentially no difference.

Reading the colors: tabpfn_threshold stays red across almost the entire grid in both directions, it helps whether imbalance is mild or severe and whether corruption is light or heavy. tabpfn_distpfn is mostly pale (little effect) across most of the grid, but turns clearly blue in the bottom-right corner of the right panel specifically, r=0.50 combined with the two highest noise levels (c=0.20, 0.30), under Direction B. That is a genuinely counter-intuitive detail: the failure shows up at r=0.50, the *easiest* imbalance condition we tested, not the hardest, meaning distpfn's specific weakness is noise interacting with near-balanced data, not imbalance severity itself.



Correction Harm Rate (left) and mean regret (right, max(0, −CG)) by method and direction, over noisy stress cells only. tabpfn_distpfn and tabpfn_distpfn_t have the highest CHR under Direction B. Regret is less uniform: tabpfn_downsample has the highest regret under Direction A despite a similar CHR to the DistPFN pair there, while tabpfn_distpfn_t has the highest regret under Direction B. tabpfn_threshold has the lowest of both in both directions.

WHEN	Drawn directly from the same CHR and mean-regret numbers already in the tables above, placed side by side specifically to compare how often a method fails against how badly it fails when it does.
WHERE	Results §4 (Direction-Dependent Failure), Figure 4.
HOW	Two grouped bar charts, one method per x-axis position, bars split by corruption direction.
WHY	CHR and regret can disagree sharply: tabpfn_downsample and tabpfn_distpfn have almost the same CHR under Direction A (20.6% vs 21.3%), but the bar chart on the right shows tabpfn_downsample's regret is more than double tabpfn_distpfn's there, the opposite of what the CHR bars alone would suggest. Seeing both side by side is what makes that reversal visible at a glance.

Break-even contamination share

Share of the 40 (dataset, r) combinations where a method's seed-averaged Correction Gain never dropped to zero or below within the tested range

Method	Dir. A	Dir. B
xgb_weighted	48%	70%

xgb_ros	55%	70%
xgb_smote	38%	78%
tabpfn_threshold	**73%**	**83%**
tabpfn_distpfn	40%	43%
tabpfn_distpfn_t	45%	40%
tabpfn_downsample	58%	70%

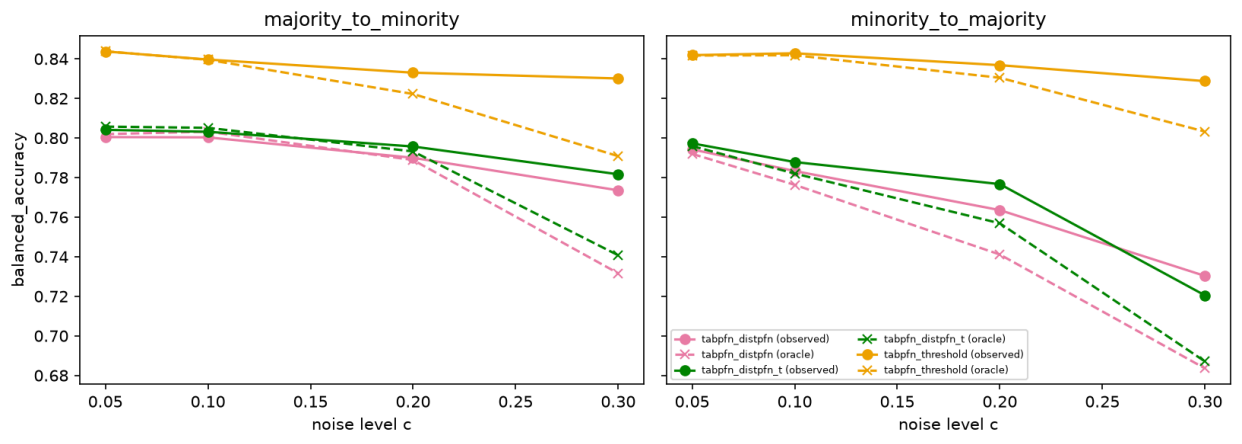
`tabpfn_threshold` leads here too, staying net-beneficial through the full tested range in the highest share of conditions in both directions. The single lowest share belongs to a different method each time, `xgb_smote` under Direction A (38%) and `tabpfn_distpfn_t` under Direction B (40%).

Prior Sensitivity Gap

Mean PSG by method and direction (negative means the oracle prior did not help, or even hurt)

Method	Dir. A	Dir. B
tabpfn_threshold	−0.013	−0.008
tabpfn_distpfn	−0.010	−0.020
tabpfn_distpfn_t	−0.010	−0.015

Mean PSG is negative for all three prior-dependent methods in both directions, the deployable, observed-prior variant slightly *outperforms* the oracle diagnostic on average rather than trailing it. This means the corrupted prior is not the main reason these corrections decline at high contamination (Discussion, Section 4).



Observed-prior (deployable, solid) versus oracle-prior (diagnostic, dashed) balanced accuracy for the three prior-dependent TabPFN methods. The oracle variant is never reliably better than the observed one, and for tabpfn_distpfn and tabpfn_distpfn_t it degrades faster at high contamination under Direction A.

WHEN	Plotted across the four non-zero contamination rates, one line per method per prior variant, so the observed and oracle trajectories for the same method can be compared directly as contamination increases.
WHERE	Results §4 (Prior Sensitivity), Figure 5.
HOW	Solid lines use the real, deployable observed prior; dashed lines use the diagnostic oracle (true, pre-corruption) prior that no real deployment could ever supply. Two panels split by corruption direction.
WHY	If the corrupted prior were the main reason these methods decline, the dashed oracle lines should sit visibly above the solid observed lines. Instead, for tabpfn_distpfn and tabpfn_distpfn_t under Direction A the dashed line drops *below* the solid one at high contamination, direct visual evidence for our most surprising finding.

Statistical significance

Friedman omnibus test, blocked by (dataset, r, c), n = 160

Family	Direction	χ^2	n (blocks)
XGBoost	Dir. A	111.1	160
XGBoost	Dir. B	198.9	160
TabPFN	Dir. A	204.0	160
TabPFN	Dir. B	332.7	160

All four significant at $p < 10^{-23}$. Blocking by (dataset, r, c) treats every tested ratio and noise level within a dataset as independent evidence, but conditions drawn from the same dataset share its feature distribution and test set, so a more conservative, dataset-level check was also run.

Friedman omnibus test, blocked by dataset alone, n = 10 (more conservative)

Family	Direction	χ^2	p
XGBoost	Dir. A	20.5	1.3×10^{-4}
XGBoost	Dir. B	21.5	8.4×10^{-5}
TabPFN	Dir. A	29.7	5.7×10^{-6}
TabPFN	Dir. B	31.0	3.0×10^{-6}

The p-values increase by several orders of magnitude under the stricter treatment, but all four remain significant well below the conventional $p < 0.05$ threshold, method identity affects balanced accuracy either way, the weaker version only overstates how strong that evidence is, not whether it exists.

Pairwise significance: which specific methods differ from which

Beyond the single "does method identity matter" question above, we also tests every specific pair of methods within a family against each other, using the Wilcoxon signed-rank test with Holm correction for multiple comparisons. This was run at both blocking levels: the original (dataset, r, c) level (n=160, 32 pairwise comparisons total) and, as an additional robustness check, the stricter dataset-only level (n=10 per family), since a significant omnibus test only guarantees *some* pair differs, not that *every specific pair* we discuss does so under the stricter standard.

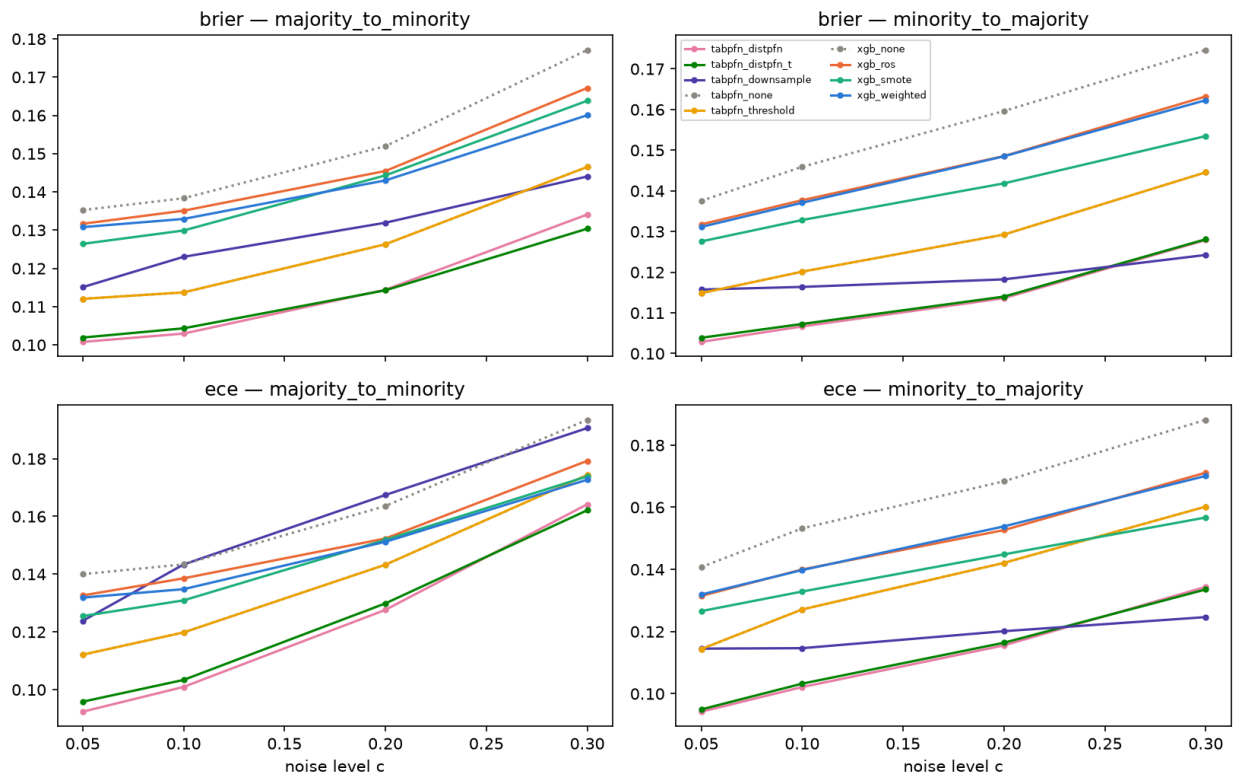
Pairwise Wilcoxon results by blocking level

Family	Comparisons tested	Significant at n=160	Significant at n=10
XGBoost	12	11	7
TabPFN	20	20	14
Total	**32**	**31**	**21**

At n=160, only one comparison is not significant (`xgb_weighted` vs. `xgb_ros` under Direction B, $p_{Holm}=0.95$). At the stricter n=10 level, only 21 of the 32 comparisons remain significant. `tabpfn_threshold` still differs significantly from every other TabPFN method in both directions at this stricter level, so the finding that it is the most robust method holds either way. The three XGBoost corrections become statistically indistinguishable from each other under Direction B (and `xgb_smote` specifically from the other two under Direction A too), though each still differs significantly from doing nothing. Two baseline-versus-correction comparisons (`tabpfn_distpfn` and `tabpfn_distpfn_t` against `tabpfn_none`, both under Direction B) also lose significance at the stricter level. None of the descriptive numbers above (CHR, CG, break-even, PSG) depend on this test in any way, this only affects which pairwise significance claims a formal test can support at n=10 rather than n=160.

Calibration reliability

Every XGBoost correction improves on its baseline's Brier score and expected calibration error (ECE) at every noise level tested, in both directions, with no exception. `tabpfn_downsample` is worse-calibrated than doing nothing on ECE at every noise level tested under Direction A, the only method in our study with that pattern on that metric, its Brier score follows the same pattern except at the highest tested contamination, where it drops back below baseline. `tabpfn_distpfn` and `tabpfn_distpfn_t` do not merely avoid backfiring on calibration, they are the two best-calibrated methods of any tested, correction or baseline, at every noise level under Direction A and all but the highest under Direction B, despite having the highest Correction Harm Rate in our study on balanced accuracy. A correction that more frequently underperforms its own baseline on discrimination can still be assigning its predictions a well-calibrated probability, discrimination and calibration are not the same failure mode.



Brier score (top) and expected calibration error (bottom) against noise level, faceted by direction. Dotted gray lines are the two uncorrected baselines. Every XGBoost correction improves on its baseline at every noise level. Within the TabPFN family, tabpfn_downsample is the only correction that crosses above its own baseline, under Direction A, while tabpfn_distpfn and tabpfn_distpfn_t stay the best-calibrated methods in the figure almost throughout.

WHEN	Tracked across all four non-zero contamination rates for every method, plotted against the two dotted gray uncorrected-baseline lines (xgb_none, tabpfn_none) for direct comparison.
WHERE	Results §4 (Calibration Reliability), Figure 6.
HOW	Four panels: Brier score and ECE (lower is better for both), each split by corruption direction, one line per method.
WHY	Balanced accuracy alone would never reveal that tabpfn_distpfn and tabpfn_distpfn_t, the two worst methods on discrimination, are simultaneously the two best-calibrated lines in every panel except one corner. Plotting calibration on its own axis is what makes that split visible instead of hidden inside a single robustness score.

5. Discussion and threats to validity

Why the simplest correction wins

`tabpfn_threshold` does less than any other correction tested, it shifts the decision boundary from 0.5 to the class prior and nothing more, yet it is the most robust method in our study on discrimination. `tabpfn_distpfn` and `tabpfn_distpfn_t` sit at the opposite end, rescaling predictions against a prior and a temperature term, and are the least robust on discrimination under Direction B specifically. This builds on prior work showing that on clean data a well-tuned classifier often gains nothing from balancing the training set (Elor & Averbuch-Elor, 2022), and shows the same pattern grows sharper, not just present, once label noise is added. There is also a structural reason beyond simplicity for its own sake: the severe-imbalance conditions this study tests combine imbalance with a real train/test prior mismatch (the test set keeps its natural class ratio while training is pushed to r), and threshold correction is nothing but a prior-shift adjustment, so it is being tested in exactly the deployment scenario its underlying idea is built for.

Direction asymmetry undercuts a single robustness score

Five of the seven methods fail more often under Direction A than Direction B, while the two DistPFN variants show the opposite pattern. A method's CHR under one direction is a poor predictor of its CHR under the other, `tabpfn_downsample`'s CHR is 20.6% under Direction A and 7.5% under Direction B, nearly a threefold difference in the same method on the same datasets. A robustness claim validated under only one corruption mechanism does not transfer to the other, which is the core reason we test every correction under both directions rather than one.

The corrupted prior is not the main failure mode

The most surprising result we found is that mean PSG is negative for all three prior-dependent TabPFN methods in both directions, the true, uncorrupted prior, something no real deployment could ever supply, does not reliably outperform the corrupted, observed one. One structural explanation: `tabpfn_threshold`, `tabpfn_distpfn`, and `tabpfn_distpfn_t` all adjust TabPFN's output probabilities after the fact, none of them touch the training rows TabPFN actually reads as its in-context prompt. Contamination happens before any correction is applied, so the mislabeled rows stay in that prompt regardless of which prior a posterior-rescaling method is given afterward, swapping in the true prior fixes the summary statistic the correction uses, but not the individual wrong examples the model has already been shown.

Accuracy and calibration answer different questions

The same two methods with the highest Correction Harm Rate in the whole study (`tabpfn_distpfn`, `tabpfn_distpfn_t`) are also its two best-calibrated methods. A system that only needs a correct label cares about balanced accuracy, and by that standard the DistPFN pair is a poor choice under contamination. A system that acts on a predicted probability (setting a risk threshold, ranking cases by urgency) cares about calibration first, and for that use case their poor CHR would be the wrong reason to rule them out. Evaluating a correction for deployment requires checking both axes separately, not inferring one from the other.

Implications

`tabpfn_threshold` stays net-beneficial through the full contamination range in 73% to 83% of tested conditions, the highest share of any method in both directions, so switching to it may fix most of what a new noise-aware training algorithm would otherwise need to solve. The DistPFN pair still fails badly enough on discrimination under high contamination that the case for joint noise-and-imbalance-aware training remains strong, for a deployment that needs a correct label specifically. A deployment that instead needs a well-calibrated probability is a different case entirely.

Threats to validity

This is a standard section in empirical research: rather than waiting for a reader or reviewer to find the weak points, we lay out ourselves the specific ways our own results could be wrong, limited, or over-generalized. It is not a confession that the study

failed, it is a calibration tool that tells the reader exactly how much to trust a finding and exactly how far it can be generalized. It splits into three standard categories, each asking a different question about the same set of results.

Internal validity — is there another explanation for our results besides the one we claim?

WHAT IT ASKS

Could something in how the study itself was built, rather than the phenomenon we say we found, explain the numbers?

IN THIS STUDY

Every TabPFN result comes from one checkpoint, one GPU backend, and one machine. Each stress cell was repeated across only three seeds, enough to reduce dependence on a single train/test split, but providing limited precision for quantities determined by whether a seed-averaged value crosses zero, specifically Correction Harm Rate and the break-even rate, a limitation our own break-even non-monotonicity finding (25.6% of curves recover after crossing zero) may partly reflect. Direction A and B are not magnitude-controlled versions of the same manipulation, they define contamination against different reference points, so comparisons between them reflect each direction's own natural severity, not an identical absolute amount of corruption. Training is deliberately pushed to each tested ratio while the test set keeps its natural class balance, so the low-ratio conditions we emphasize combine imbalance with train/test prior shift rather than isolating imbalance alone.

External validity — how far do these findings generalize beyond exactly what we tested?

WHAT IT ASKS

Would the same conclusions hold on a different scale, a different data modality, or a different kind of noise than the ones actually tested?

IN THIS STUDY

The ten datasets range from 270 to 17,898 raw rows, but the controlled training set every method actually fits on tops out at 2,622 rows even for the largest dataset, staying within TabPFN's intended small-tabular-data regime, not the scale where gradient-boosted methods are typically deployed. Every dataset audited is a binary-classification problem, whether the same patterns extend to multi-class imbalance is untested. Both corruption mechanisms are class-conditional (the flip probability depends only on the true class), not instance-dependent, and only one direction is ever applied at a time, never a mixture of both within the same corrupted training set. Contamination was tested only up to c=0.30, never beyond it.

Construct validity — are we actually measuring the thing we claim to be measuring?

WHAT IT ASKS

Do our metrics and diagnostic constructs validly capture the concept they are meant to represent, or could they be quietly measuring something narrower or different?

IN THIS STUDY

Correction Gain, Correction Harm Rate, Regret, and the break-even rate are original to our work and reported only on balanced accuracy, not the other nine metrics tracked, so "robust by CHR" does not automatically mean robust by every measure. The break-even rate assumes a correction's gain declines monotonically as contamination rises, an assumption our own data does not fully support. The Prior Sensitivity Gap's oracle variant assumes access to the true, pre-corruption class prior, something no real deployment has, it is a diagnostic construct for isolating a cause, not a claim about achievable real-world performance.

6. **Related work — systematic literature review table**

Every substantive prior work our Related Work section discusses and positions itself against, in the standard six-column literature-review format. Tool citations (scikit-learn, XGBoost, imbalanced-learn) and pure statistical-method citations (the Friedman test, Wilcoxon test, Holm correction) are omitted here, since they describe methods we *use*, not prior findings we are arguing against or building on (see the References in Section 7 for those). The "Possible Gap / So What?" column states, in each case, what we specifically do that the cited work does not.

PAPER / CITATION	RESEARCH PROBLEM / QUESTION	METHOD / DATA / SYSTEM / APPROACH	KEY FINDING / CONTRIBUTION	LIMITATION	POSSIBLE GAP / "SO WHAT?"
Hollmann et al., 2023 — TabPFN	Fast tabular classification without per-dataset training	Transformer pretrained once on millions of synthetic datasets; in-context learning, one forward pass per test set	Matches or exceeds tuned gradient-boosted ensembles on small tabular datasets with no per-dataset fitting	Behavior under a training-time class distribution that differs from the pretraining prior (i.e. imbalance) is largely unexamined	We stress-test TabPFN and the corrections built on it under directed imbalance and label noise, exactly the untested condition
Hollmann et al., 2025 — TabPFN v2	Scaling the TabPFN architecture and pretraining prior	Scaled version of the same in-context, prior-data-fitted architecture	Matches or exceeds tuned GBMs on datasets up to 10,000 rows, orders of magnitude faster	Same imbalance/noise-robustness gap as v1	Same as above, this is the checkpoint version we actually audit
Hu & Ghelichi, 2026	Does TabPFN's internal mechanism degrade under label noise?	Analysis of TabPFN's internal attention patterns under synthetic label-quality degradation	Attention stays structured and ROC-AUC stays high with no correction applied at all	Examines the base model's internals only, not the correction methods practitioners actually deploy on top of it	We audit the deployed correction methods themselves (threshold, DistPFN, downsampling), not the uncorrected model's internals
Lee, 2026 — DistPFN	Correcting TabPFN's predictions for prior/label shift without retraining	Rescales TabPFN's output probabilities against a class prior; adds a temperature-scaled variant (DistPFN-T)	First prior-adjustment correction brought to an in-context tabular foundation model	The published DistPFN-T reference implementation has a shape-error bug (found during our own verification against it) that silently fails for two-class problems	We are the first to test DistPFN and DistPFN-T's robustness under directed label contamination rather than only on clean data
Saerens et al., 2002	Adjusting a classifier's output probabilities to a new target class prior without retraining	Analytical posterior-adjustment formula for conventional probabilistic classifiers	Classical formalization of prior adjustment, decades before tabular foundation models existed	Formulated for conventional classifiers, not in-context foundation models	Basis for our threshold correction and DistPFN, we are the first to test the idea under label noise on TabPFN specifically
Chawla et al., 2002 — SMOTE	Correcting class imbalance by synthesizing new minority examples	Interpolates between a minority row and one of its k nearest minority neighbors	Still one of the most widely adopted imbalance corrections nearly 25 years after publication	Assumes the labels used to find "minority neighbors" are themselves trustworthy; not designed for or evaluated under label noise	We test standard SMOTE specifically under directed, asymmetric label contamination
Elkan, 2001	Correcting imbalance without altering the training distribution	Reweights the loss so minority-class errors cost more, in proportion to the observed class ratio	Cost-sensitive framing implemented directly in XGBoost's `scale_pos_weight`	The correction depends entirely on the observed class ratio being accurate	We test exactly what happens once that observed ratio is itself corrupted by label noise
Elor & Averbuch-Elor, 2022	Does balancing the training set actually help, even when every label is trustworthy?	Empirical comparison of balanced vs. imbalanced training across classifiers and hyperparameters	With a sufficiently strong classifier, balancing often provides no benefit and can be actively counterproductive	Tested only on clean, uncorrupted labels	We show the same "correction may not help" pattern grows sharper, not just present, once label noise is added on top
Natarajan et al., 2013	Theoretical conditions for learning under class-conditional label noise	Theoretical formalization and consistency proofs	Conditions under which a consistent classifier can still be recovered despite class-conditional noise	General classifier theory, not tied to any specific imbalance-correction method	Our two directed corruption mechanisms operationalize this framework specifically against imbalance corrections
Scott et al., 2013	How does asymmetric, class-dependent noise differ from symmetric noise?	Theoretical identifiability analysis	The direction and structure of label corruption, not just its rate, changes how a classifier should be corrected	Theoretical; not evaluated against imbalance-correction behavior	Motivates our two distinct, asymmetric corruption mechanisms in place of a symmetric random flip
Frénay & Verleysen, 2014	Survey of label-noise sources, taxonomies, and effects (pre-deep-learning)	Literature survey	Taxonomy of noise sources and types and their effects on classifier robustness	Treats label noise as a phenomenon to correct or tolerate during training; does not examine its interaction with imbalance correction	We study exactly that unexamined interaction

Song et al., 2022	Survey of deep-learning approaches to noisy-label robustness	Literature survey (loss correction, sample selection, label correction)	Catalog of noise-robust deep-training approaches	Same as above; no examination of interaction with imbalance correction	Same as above
Van Hulse & Khoshgoftaar, 2009	How do sampling techniques and learners interact with class noise under imbalance?	Large-scale empirical study: over two million classifiers, seven datasets, eleven learners, seven sampling techniques, varying noise location and level	Which class receives the noise materially affects learning/sampling behavior; imbalance mitigation interacts strongly with noise in classical settings	Classical learners and samplers only; training-set size not held fixed across imbalance ratios, so ratio and size are confounded	We extend to tabular foundation models and control training-set size so ratio and sample size are never confounded
Chen et al., 2021 — RSMOTE	Making SMOTE itself robust to label noise	Self-adaptive, density-weighted SMOTE variant; no separate noise filter or extra parameter needed	New noise-robust resampling algorithm	Proposes a new algorithm rather than auditing whether the standard, already-deployed SMOTE remains usable as is	We audit whether the standard correction remains net-beneficial as deployed, rather than proposing a replacement
Li et al., 2024 — WRND	General framework for noise-robust oversampling	Natural-neighbor noise/outlier filtering combined with relative-neighborhood-density weighting	Framework compatible with most SMOTE-based algorithms, improves their noise robustness	Same as RSMOTE: a new algorithm, not an audit of already-deployed methods	Same as above
Koziarski et al., 2020	Jointly cleaning and resampling multi-class imbalanced, noisy data	Combined cleaning-and-resampling algorithm (CCR)	New algorithm for multi-class imbalanced data known to contain label noise	New training-time algorithm; multi-class scope; does not audit already-deployed corrections	Our binary-classification, audit-not-propose scope, extended to tabular foundation models
Khanal et al., 2024	Jointly handling imbalance and label noise in medical image classification	Active learning combined with noisy-label training	Active label-refinement approach for imbalanced, noisy medical image tasks	Domain-specific to medical images; proposes a new training method rather than auditing deployed corrections	Our tabular, audit-focused approach covers a different, unaddressed setting and data modality
Liu et al., 2024	Loss-based sample selection for noisy-label training is itself biased against minority classes	Class-balanced sample selection with confidence-based augmentation	Under-learned tail classes and noise-corrupted classes both show high training loss and are hard to tell apart; new procedure corrects the resulting selection bias	Proposes a new training-time algorithm rather than auditing existing, widely-deployed corrections	Same audit-not-propose gap we address, applied to tabular foundation-model corrections specifically

7. References

Every work we cite, alphabetical by first author. Datasets are listed separately at the end.

Methods, models, and prior work

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

Chen, B., Xia, S., Chen, Z., Wang, B., & Wang, G. (2021). RSMOTE: A self-adaptive robust SMOTE for imbalanced problems with label noise. *Information Sciences*, 553, 397–428.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 785–794.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.

Elkan, C. (2001). The foundations of cost-sensitive learning. *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, 973–978.

Elor, Y., & Averbuch-Elor, H. (2022). To SMOTE, or not to SMOTE?

Frénay, B., & Verleysen, M. (2014). Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5), 845–869.

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200), 675–701.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 1321–1330.

Hollmann, N., Müller, S., Eggensperger, K., & Hutter, F. (2023). TabPFN: A transformer that solves small tabular classification problems in a second. *International Conference on Learning Representations (ICLR)*.

Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., et al. (2025). Accurate predictions on small data with a tabular foundation model. *Nature*, 637, 319–326.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.

Hu, J., & Ghelichi, M. (2026). Noise immunity in in-context tabular learning: An empirical robustness analysis of TabPFN's attention mechanisms. arXiv:2604.04868.

Khanal, B., Dai, T., Bhattarai, B., & Linte, C. (2024). Active label refinement for robust training of imbalanced medical image classification tasks in the presence of high label noise. *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, 37–47.

Koziarski, M., Woźniak, M., & Krawczyk, B. (2020). Combined cleaning and resampling algorithm for multi-class imbalanced data with label noise. *Knowledge-Based Systems*, 204, 106223.

Lee, S. (2026). Mitigating label shift in tabular in-context learning via test-time posterior adjustment. *International Conference on Machine Learning (ICML)*. arXiv:2605.04363.

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets. *Journal of Machine Learning Research*, 18(17), 1–5.

Li, M., Zhou, H., Liu, Q., Gong, X., & Wang, G. (2024). WRND: A weighted oversampling framework with relative neighborhood density for imbalanced noisy classification. *Expert Systems with Applications*, 241, 122593.

Liu, H., Sheng, M., Sun, Z., Yao, Y., Hua, X.-S., & Shen, H.-T. (2024). Learning with imbalanced noisy data by preventing bias in sample selection. *IEEE Transactions on Multimedia*.

Natarajan, N., Dhillon, I. S., Ravikumar, P., & Tewari, A. (2013). Learning with noisy labels. *Advances in Neural Information Processing Systems (NeurIPS)*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Saerens, M., Latinne, P., & Decaestecker, C. (2002). Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1), 21–41.

Scott, C., Blanchard, G., & Handy, G. (2013). Classification with asymmetric label noise: Consistency and maximal denoising. *Journal of Machine Learning Research Workshop and Conference Proceedings (COLT)*, 30, 489–511.

Song, H., Kim, M., Park, D., Shin, Y., & Lee, J.-G. (2022). Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11), 8135–8153.

Van Hulse, J., & Khoshgoftaar, T. M. (2009). Knowledge discovery from imbalanced and noisy data. *Data & Knowledge Engineering*, 68(12), 1513–1542.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.

Datasets (UCI Machine Learning Repository)

Kelly, M., Longjohn, R., & Nottingham, K. The UCI Machine Learning Repository. https://archive.ics.uci.edu/

Lohweg, V. (2012). Banknote Authentication [Dataset]. https://archive.ics.uci.edu/dataset/267

Yeh, I-C. (2008). Blood Transfusion Service Center [Dataset]. https://archive.ics.uci.edu/dataset/176

Grömping, U. (2019). South German Credit Data: Correcting a widely used data set. https://archive.ics.uci.edu/dataset/522

Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). Spambase [Dataset]. https://archive.ics.uci.edu/dataset/94

Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1993). Breast Cancer Wisconsin (Diagnostic) [Dataset]. https://archive.ics.uci.edu/dataset/17

Sigillito, V., Wing, S., Hutton, L., & Baker, K. (1989). Ionosphere [Dataset]. https://archive.ics.uci.edu/dataset/52

Haberman, S. (1999). Haberman's Survival [Dataset]. https://archive.ics.uci.edu/dataset/43

Statlog (Heart) [Dataset]. https://archive.ics.uci.edu/dataset/145

Lyon, R. (2015). HTRU2 [Dataset]. https://archive.ics.uci.edu/dataset/372

Mansouri, K., Ringsted, T., Ballabio, D., Todeschini, R., & Consonni, V. (2013). QSAR Biodegradation [Dataset]. https://archive.ics.uci.edu/dataset/254